Entity Framework Core

1. ORM и Entity Framework Core

ORM (Object-Relational Mapping) — подход, при котором таблицы реляционной базы данных сопоставляются с классами языка программирования, а строки таблиц — с экземплярами этих классов.

Вместо того чтобы думать в терминах таблиц, строк и столбцов, разработчик работает с обычными объектами C#. ORM берёт на себя преобразование между объектной моделью приложения и реляционной моделью базы данных.

Реляционная база данных	Объектная модель C#
Таблица	Класс
Строка	Экземпляр объекта
Столбец	Свойство
Внешний ключ	Навигационное свойство
SQL-запрос	LINQ-выражение

Entity Framework Core (EF Core) — официальная ORM-библиотека для платформы .NET, разрабатываемая и поддерживаемая Microsoft. Она позволяет работать с базой данных через объектную модель: автоматически генерирует SQL-запросы, управляет соединениями и отслеживает изменения объектов.

EF Core поддерживает множество СУБД через систему провайдеров — тот же код работает с SQLite, PostgreSQL, MS SQL Server и другими базами данных, меняется только строка подключения и пакет провайдера.

Преимущества и ограничения

Преимущества:

- **Меньше кода.** Стандартные CRUD-операции — несколько строк без SQL вручную.
- **Типобезопасность.** LINQ-запросы проверяются компилятором. Ошибка в имени свойства обнаружится при компиляции, а не во время выполнения.
- **Управление схемой через миграции.** Структура базы данных описывается в коде и эволюционирует вместе с моделями — не нужно писать SQL-скрипты для изменения схемы вручную.
- **Отслеживание изменений.** EF Core автоматически определяет, какие объекты изменились, и генерирует только необходимые SQL-команды при сохранении.
- **Независимость от СУБД.** Один и тот же код работает с разными базами данных.

Ограничения:

- **Скрывает реальный SQL.** Разработчик не всегда видит, какой запрос генерируется. Без понимания SQL легко написать код, который выглядит чистым, но выполняет десятки лишних запросов к базе.
- **Генерирует неоптимальные запросы.** Для сложных аналитических запросов или специфичных для СУБД функций написанный вручную SQL часто эффективнее.
- **Добавляет накладные расходы.** Отслеживание изменений, материализация объектов и другие механизмы EF Core потребляют ресурсы — для критически нагруженных участков кода это может быть значимо.

Code First и Database First

При работе с EF Core существует два подхода.

Code First — разработка начинается с C#-кода. Разработчик описывает модели данных в виде классов, EF Core на их основе создаёт и изменяет схему базы данных через механизм миграций. Весь контроль над схемой — в коде.

```
Описываем классы → Создаём миграцию → Применяем к БД → Меняем классы → Новая
миграция → ...
```

Database First — разработка начинается с базы данных. Разработчик проектирует схему вручную в SQL, затем EF Core генерирует классы моделей и контекст на основе существующей структуры командой `dotnet ef dbcontext scaffold`. При изменении схемы команду нужно запускать повторно.

```
Создаём схему в SQL → Генерируем модели → Работаем с кодом → Меняем БД →
Повторная генерация
```

	Code First	**Database First**
Источник схемы	C#-код	База данных
Управление схемой	Через миграции	Вручную в SQL
История изменений	В файлах миграций (в репозитории)	Вне проекта
Когда использовать	Новый проект	Существующая БД

В этой лекции мы используем **Code First** — он лучше подходит для новых проектов и учебных сценариев.

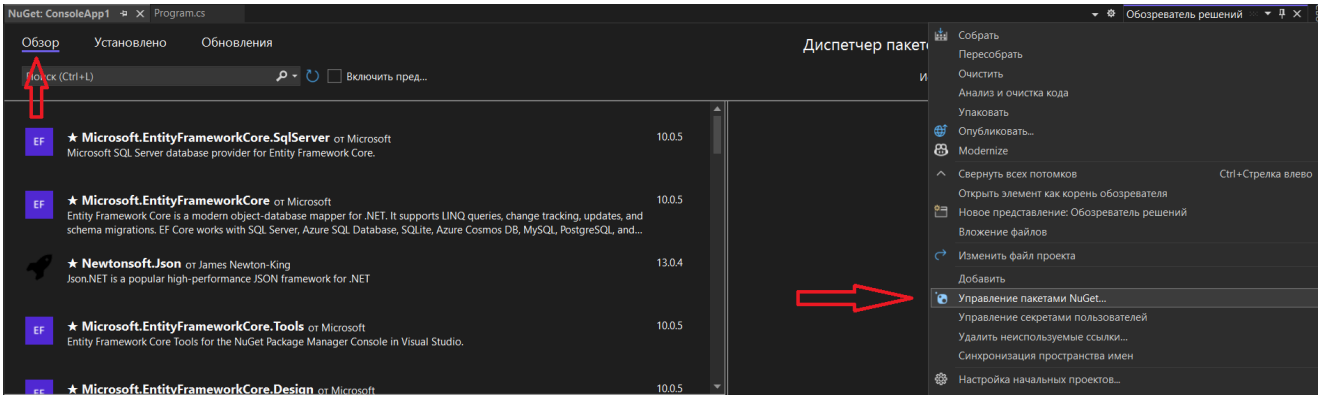
2. Установка и настройка

Для работы с SQLite нужно три пакета. `Microsoft.EntityFrameworkCore` — ядро EF Core. Содержит базовые классы: `DbContext`, `DbSet`, механизм отслеживания изменений, LINQ-провайдер.

`Microsoft.EntityFrameworkCore.Sqlite` — провайдер для SQLite. Содержит реализацию подключения и генератор SQL-запросов, специфичных для этой СУБД. Автоматически подтягивает `Microsoft.Data.Sqlite`.

`Microsoft.EntityFrameworkCore.Tools` — инструменты командной строки для работы с миграциями.

Для того, чтобы установить пакеты нажмите ПКМ по проекту в обозревателе решений и выберите *Управление пакетами NuGet*:



Либо через консоль:

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
```

Установка dotnet-ef

Команды `dotnet ef` предоставляются отдельным глобальным инструментом. Устанавливается один раз на всю систему:

```
dotnet tool install --global dotnet-ef
```

Проверка установки:

```
dotnet ef --version
```

Структура проекта

Рекомендуется разделять файлы по назначению с первых шагов — это упрощает ориентацию по мере роста проекта:

```
MyApp/
├── Models/
│   ├── User.cs            — сущности (модели данных)
│   └── Order.cs
├── Data/
│   └── AppDbContext.cs    — контекст базы данных
├── Migrations/            — создаётся автоматически командой dotnet ef
├── Services/
│   ├── UserRepository.cs  — репозитории для работы с БД
│   └── OrderRepository.cs
├── Program.cs
└── MyApp.csproj
```

Первая сущность и DbContext

Сущность — это обычный класс C#, свойства которого соответствуют столбцам таблицы:

```
// Models/Book.cs
public class Book
{
    public int    Id     { get; set; }
    public string Title  { get; set; } = "";
    public string Author { get; set; } = "";
    public int    Year   { get; set; }
    public double Price  { get; set; }
}
```

EF Core применит соглашения по умолчанию: класс `Book` → таблица `Books`, свойство `Id` → первичный ключ с автоинкрементом, остальные свойства → столбцы.

`DbContext` — центральный класс EF Core. Он управляет подключением к базе, отслеживает изменения объектов и знает, какие таблицы существуют. В приложении всегда создаётся собственный класс, наследующий `DbContext`:

```
// Data/AppDbContext.cs
using Microsoft.EntityFrameworkCore;

public class AppDbContext : DbContext
{
    // Каждый DbSet соответствует одной таблице
    public DbSet<Book> Books { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseSqlite("Data Source=library.db");
}
```

`OnConfiguring` — метод, в котором настраивается подключение. `UseSqlite` принимает строку подключения — путь к файлу базы данных.

`DbContext` должен иметь **короткий жизненный цикл** — создаётся для одной логической операции и сразу освобождается через `using`.

Создание базы данных

После того как сущности и контекст определены, нужно создать базу данных. Есть два способа.

`EnsureCreated` — быстрый способ для учебных и тестовых проектов. Создаёт базу данных и все таблицы на основе текущих моделей, если база ещё не существует:

```
using var context = new AppDbContext();
await context.Database.EnsureCreatedAsync();
```

Через миграции — основной подход для реальных приложений. Миграции отслеживают историю изменений схемы и позволяют безопасно эволюционировать структуру базы данных. Подробно рассматривается в разделе 6.

Database First

Мы используем Code First, но стоит знать об альтернативном подходе — он встречается в реальных проектах.

Database First применяется, когда база данных уже существует — например, унаследованная система, которую нужно поддерживать, или проект, где схемой управляет отдельный администратор БД. В этом случае классы моделей и контекст не пишутся вручную, а **генерируются автоматически** из существующей схемы командой `dotnet ef dbcontext scaffold`, в этой команде нужно указать:

- Строку подключения к БД
- Провайдер БД
  и опционально:
- `--output-dir`: название папки в которой будут созданы классы моделей
- `--context`: название для класса `DbContext`
  Например:

```
dotnet ef dbcontext scaffold "Data Source=library.db" \
    Microsoft.EntityFrameworkCore.Sqlite \
    --output-dir Models \
    --context AppDbContext
```

EF Core прочитает схему базы данных и создаст файлы автоматически.

3. Модели и конфигурация

Модели — это C#-классы, которые описывают структуру данных приложения. EF Core использует их для создания таблиц, генерации SQL-запросов и маппинга результатов. Конфигурация определяет, как именно классы и их свойства сопоставляются с таблицами, столбцами, ключами и ограничениями.

Соглашения по умолчанию

EF Core может работать практически без явной конфигурации благодаря соглашениям, которые применяются автоматически:

- класс `Book` → таблица `Books`
- свойство `Id` или `BookId` → первичный ключ с автоинкрементом
- `string` → столбец `TEXT NOT NULL`
- `string?` → столбец `TEXT NULL`
- `int`, `double`, `decimal` → соответствующий числовой тип, `NOT NULL`
- навигационное свойство + свойство `<Тип>Id` → внешний ключ

Простейшая сущность без какой-либо конфигурации уже работает корректно:

```
public class Book
{
    public int    Id     { get; set; }   // → PK, AUTOINCREMENT
    public string Title  { get; set; } = "";  // → TEXT NOT NULL
    public string Author { get; set; } = "";  // → TEXT NOT NULL
    public int    Year   { get; set; }   // → INTEGER NOT NULL
    public double Price  { get; set; }   // → REAL NOT NULL
}
```

Когда соглашений недостаточно — нестандартные имена таблиц, уникальные индексы, ограничения, значения по умолчанию — используется явная конфигурация.

Три способа конфигурации

EF Core предоставляет три уровня конфигурации, которые можно комбинировать.

Соглашения — работают автоматически, ничего писать не нужно.

Атрибуты (Data Annotations) — указываются прямо на классе или свойстве.

Fluent API — настраивается в методе `OnModelCreating` контекста или в отдельных классах конфигурации.

Если одно и то же свойство настроено несколькими способами, приоритет такой: Fluent API > атрибуты > соглашения.

Конфигурация через атрибуты

Атрибуты удобны для простых случаев — они видны прямо в классе и не требуют отдельного файла:

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("books")]                        // явное имя таблицы
public class Book
{
    [Key]                               // первичный ключ
    public int Id { get; set; }

    [Required]                          // NOT NULL
    [MaxLength(200)]                    // максимальная длина
    public string Title { get; set; } = "";

    [Required]
    [MaxLength(100)]
    public string Author { get; set; } = "";

    public int Year  { get; set; }

    [Column("retail_price")]            // явное имя столбца
    public double Price { get; set; }

    [NotMapped]                         // свойство не сохраняется в БД
    public string DisplayName => $"{Title} ({Author}, {Year})";
}
```

Основные атрибуты:

Атрибут	Назначение
`[Key]`	Первичный ключ
`[Required]`	NOT NULL
`[MaxLength(n)]`	Максимальная длина строки
`[Column("name")]`	Явное имя столбца
`[Table("name")]`	Явное имя таблицы
`[NotMapped]`	Не сохранять в БД
`[ForeignKey("prop")]`	Явное указание внешнего ключа

Атрибуты не покрывают всех возможностей конфигурации. Уникальные индексы, CHECK-ограничения, составные ключи, точная настройка связей требует Fluent API.

Fluent API

Fluent API — наиболее полный способ конфигурации. Настройка выполняется в методе `OnModelCreating` и не загромождает сами классы сущностей:

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Book>(entity =>
    {
        entity.ToTable("books");                        // имя таблицы

        entity.Property(b => b.Title)
              .IsRequired()                             // NOT NULL
              .HasMaxLength(200);

        entity.Property(b => b.Author)
              .IsRequired()
              .HasMaxLength(100);

        entity.Property(b => b.Price)
              .HasColumnName("retail_price")            // имя столбца
              .HasDefaultValue(0.0);                    // значение по
умолчанию

        entity.HasIndex(b => b.Title);                  // индекс для ускорения
поиска

        // CHECK-ограничение: цена не может быть отрицательной
        entity.ToTable(t => t.HasCheckConstraint("CK_Book_Price",
"retail_price >= 0"));
    });
}
```

Полный список возможностей Fluent API существенно шире атрибутов. Наиболее часто используемые методы:

Метод	Назначение
`.ToTable("name")`	Явное имя таблицы
`.HasKey(x => x.Id)`	Первичный ключ
`.Property(...).IsRequired()`	NOT NULL
`.Property(...).HasMaxLength(n)`	Максимальная длина
`.Property(...).HasColumnName("name")`	Явное имя столбца
`.Property(...).HasDefaultValue(val)`	Значение по умолчанию (C#)
`.Property(...).HasDefaultValueSql("...")`	Значение по умолчанию (SQL)

Метод	Назначение
`.Property(...).HasColumnType("TEXT")`	Явный тип столбца
`.HasIndex(x => x.Prop)`	Индекс
`.HasIndex(...).IsUnique()`	Уникальный индекс
`.HasCheckConstraint("name", "sql")`	CHECK-ограничение

IEntityTypeConfiguration — конфигурация в отдельных классах

Когда сущностей много, весь Fluent API в одном методе `OnModelCreating` превращается в огромный нечитаемый блок. Правильное решение — вынести конфигурацию каждой сущности в отдельный класс, реализующий `IEntityTypeConfiguration<T>`:

```
// Data/Configurations/BookConfiguration.cs
public class BookConfiguration : IEntityTypeConfiguration<Book>
{
    public void Configure(EntityTypeBuilder<Book> builder)
    {
        builder.ToTable("books");

        builder.Property(b => b.Title)
               .IsRequired()
               .HasMaxLength(200);

        builder.Property(b => b.Author)
               .IsRequired()
               .HasMaxLength(100);

        builder.Property(b => b.Price)
               .HasColumnName("retail_price")
               .HasDefaultValue(0.0);

        builder.HasIndex(b => b.Title);

        builder.ToTable(t => t.HasCheckConstraint("CK_Book_Price",
"retail_price >= 0"));
    }
}
```

Для каждой сущности создается по одному такому классу. В `AppDbContext` они подключаются одним вызовом — `ApplyConfigurationsFromAssembly` автоматически найдёт и применит все классы `IEntityTypeConfiguration` в сборке:

```
public class AppDbContext : DbContext
{
    public DbSet<Book> Books { get; set; }

    protected override void OnConfiguring(DbContextOptionsBuilder options)
        => options.UseSqlite("Data Source=library.db");

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {

modelBuilder.ApplyConfigurationsFromAssembly(typeof(AppDbContext).Assembly);
    }
}
```

Структура проекта с конфигурациями:

```
Data/
├── AppDbContext.cs
└── Configurations/
    ├── BookConfiguration.cs
    ├── AuthorConfiguration.cs
    └── ...
```

Каждый файл конфигурации отвечает за одну сущность, легко находится и редактируется независимо от остальных. Это стандартный подход в реальных проектах.

4. Связи между сущностями

Реляционные базы данных строятся вокруг связей между таблицами. В EF Core эти связи описываются через навигационные свойства и внешние ключи прямо в моделях. EF Core автоматически использует их при генерации JOIN-запросов, управлении целостностью данных и загрузке связанных объектов.

Навигационные свойства и внешние ключи

Связь в EF Core состоит из двух элементов.

Внешний ключ — обычное свойство, которое хранит идентификатор связанной записи. Обеспечивает целостность на уровне базы данных.

Навигационное свойство — свойство, которое ссылается на связанный объект или коллекцию объектов. Используется в коде для перехода между связанными сущностями.

Оба элемента используются вместе: внешний ключ — для фильтрации и целостности, навигационное свойство — для работы с объектами в коде и генерации JOIN.

Один-ко-многим

Самый распространённый тип связи. Один автор написал много книг, одна книга принадлежит одному автору.

Модели:

```
public class Author
{
    public int    Id      { get; set; }
    public string Name    { get; set; } = "";
    public string Country { get; set; } = "";
    // навигационное свойство
    public ICollection<Book> Books { get; set; } = new List<Book>();
}

public class Book
{
    public int    Id       { get; set; }
    public int    AuthorId { get; set; }  // внешний ключ
    public string Title    { get; set; } = "";
    public int    Year     { get; set; }
    public double Price    { get; set; }
    // навигационное свойство
    public Author Author { get; set; } = null!;
}
```

EF Core распознаёт связь по соглашению — если свойство называется `AuthorId` и есть навигационное свойство `Author`, внешний ключ подхватывается автоматически. Явная конфигурация нужна, когда имена не совпадают с соглашениями или требуется настроить `OnDelete`.

Конфигурация:

```
modelBuilder.Entity<Book>()
    .HasOne(b => b.Author)           // Book имеет одного Author
    .WithMany(a => a.Books)          // Author имеет много Books
    .HasForeignKey(b => b.AuthorId)  // внешний ключ — в Book
    .OnDelete(DeleteBehavior.Restrict);
```

Один-к-одному

Каждая запись одной таблицы связана ровно с одной записью другой. Например, у каждой книги есть детальное описание, которое хранится отдельно — чтобы не загружать тяжёлые данные при получении списка книг.

Модели:

```
public class Book
{
    public int    Id    { get; set; }
    public string Title { get; set; } = "";
    public double Price { get; set; }

    public BookDetails? Details { get; set; }
}

public class BookDetails
{
    public int     Id          { get; set; }
    public int     BookId      { get; set; }  // внешний ключ
    public string? Description { get; set; }
    public string? CoverUrl    { get; set; }
    public int     Pages       { get; set; }

    public Book Book { get; set; } = null!;
}
```

Конфигурация:

```
modelBuilder.Entity<Book>()
    .HasOne(b => b.Details)
    .WithOne(d => d.Book)
    .HasForeignKey<BookDetails>(d => d.BookId)  // внешний ключ всегда
указывается явно
    .OnDelete(DeleteBehavior.Cascade);
```

В отличие от один-ко-многим, здесь EF Core не может угадать, на какой стороне находится внешний ключ — поэтому `HasForeignKey<T>` с явным указанием типа обязателен.

Многие-ко-многим

Одна книга может относиться ко многим жанрам, один жанр охватывает много книг.

Без дополнительных данных

EF Core автоматически создаёт промежуточную таблицу — её не нужно описывать в коде:

```
public class Book
{
    public int    Id     { get; set; }
    public string Title  { get; set; } = "";

    public ICollection<Genre> Genres { get; set; } = new List<Genre>();
}

public class Genre
{
    public int    Id   { get; set; }
    public string Name { get; set; } = "";

    public ICollection<Book> Books { get; set; } = new List<Book>();
}
```

```
modelBuilder.Entity<Book>()
    .HasMany(b => b.Genres)
    .WithMany(g => g.Books);
```

EF Core создаст промежуточную таблицу автоматически:

```
CREATE TABLE "BookGenre" (
    "BooksId"  INTEGER NOT NULL,
    "GenresId" INTEGER NOT NULL,
    PRIMARY KEY ("BooksId", "GenresId")
);
```

С дополнительными данными

Если нужно хранить информацию о самой связи — например, дату добавления книги в коллекцию пользователя и его оценку — промежуточная сущность описывается явно:

```
public class User
{
    public int    Id       { get; set; }
    public string Name     { get; set; } = "";

    public ICollection<UserBook> UserBooks { get; set; } = new
List<UserBook>();
}

public class Book
{
    public int    Id    { get; set; }
    public string Title { get; set; } = "";

    public ICollection<UserBook> UserBooks { get; set; } = new
List<UserBook>();
}

public class UserBook  // промежуточная сущность
{
    public int      UserId  { get; set; }
    public int      BookId  { get; set; }
    public DateTime AddedAt { get; set; } = DateTime.UtcNow;
    public int?     Rating  { get; set; }

    public User User { get; set; } = null!;
    public Book Book { get; set; } = null!;
}
```

```
// Составной первичный ключ — пара (UserId, BookId) уникальна
modelBuilder.Entity<UserBook>()
    .HasKey(ub => new { ub.UserId, ub.BookId });

modelBuilder.Entity<UserBook>()
    .HasOne(ub => ub.User)
    .WithMany(u => u.UserBooks)
    .HasForeignKey(ub => ub.UserId);

modelBuilder.Entity<UserBook>()
    .HasOne(ub => ub.Book)
    .WithMany(b => b.UserBooks)
    .HasForeignKey(ub => ub.BookId);
```

Каскадные операции

`OnDelete` определяет, что происходит с зависимыми записями при удалении родительской:

```
// Cascade — удаление Author удаляет все его Books
modelBuilder.Entity<Book>()
    .HasOne(b => b.Author)
    .WithMany(a => a.Books)
    .OnDelete(DeleteBehavior.Cascade);
// Restrict — нельзя удалить Author, пока у него есть Books
modelBuilder.Entity<Book>()
    .HasOne(b => b.Author)
    .WithMany(a => a.Books)
    .OnDelete(DeleteBehavior.Restrict);
// SetNull — при удалении Author внешний ключ в Book становится NULL
// AuthorId должен быть int?, а не int
modelBuilder.Entity<Book>()
    .HasOne(b => b.Author)
    .WithMany(a => a.Books)
    .OnDelete(DeleteBehavior.SetNull);
```

Поведение	Когда использовать
`Cascade`	Зависимые данные не имеют смысла без родителя (детали книги, позиции заказа)
`Restrict`	Удаление должно быть явно заблокировано (нельзя удалить автора, пока есть книги)
`SetNull`	Связь необязательная, зависимая запись может существовать без родителя

5. CRUD

Для запросов к БД используются методы, похожие на методы LINQ.

Create

Для добавления одной сущности используется `Add()`:

```
using var context = new AppDbContext();

var author = new Author { Name = "Булгаков", Country = "Россия" };

context.Authors.Add(author);          // State → Added
await context.SaveChangesAsync();     // INSERT INTO Authors ...

// После сохранения Id заполнен автоматически
Console.WriteLine($"Создан автор Id = {author.Id}");
```

`AddRange` добавляет несколько записей за один вызов — все сохраняются одной транзакцией:

```
using var context = new AppDbContext();

context.Authors.AddRange(
    new Author { Name = "Толстой",      Country = "Россия" },
    new Author { Name = "Достоевский", Country = "Россия" },
    new Author { Name = "Тургенев",     Country = "Россия" }
);
await context.SaveChangesAsync();
```

При добавлении сущности со связанными объектами EF Core автоматически проставляет внешние ключи:

```
using var context = new AppDbContext();

var author = new Author
{
    Name    = "Булгаков",
    Country = "Россия",
    Books   = new List<Book>
    {
        new Book { Title = "Мастер и Маргарита", Year = 1967, Price = 650 },
        new Book { Title = "Белая гвардия",      Year = 1925, Price = 490 }
    }
};
```

```
context.Authors.Add(author);
await context.SaveChangesAsync();
// Author и оба Book сохраняются вместе, AuthorId проставляется
автоматически

Console.WriteLine($"Автор Id={author.Id}, книг: {author.Books.Count}");
```

SaveChangesAsync и атомарность

`SaveChangesAsync` применяет все накопленные изменения к базе данных. До его вызова любые добавления, изменения и удаления существуют только в памяти контекста.

```
using var context = new AppDbContext();

// Всё это — только в памяти
context.Authors.Add(new Author { Name = "Толстой",     Country = "Россия"
});
context.Authors.Add(new Author { Name = "Достоевский", Country = "Россия"
});
context.Authors.Add(new Author { Name = "Чехов",       Country = "Россия"
});

// Только здесь данные записываются в базу — одной транзакцией
int saved = await context.SaveChangesAsync();
Console.WriteLine($"Сохранено: {saved}"); // 3
```

Важно: один вызов `SaveChangesAsync` выполняется **в рамках одной транзакции**. Если при сохранении нескольких объектов что-то пойдёт не так — все изменения откатятся. Это обеспечивает атомарность без явного управления транзакцией в простых сценариях.

Read

Чтение — самая богатая по возможностям операция. EF Core транслирует LINQ-запросы в SQL и выполняет их на стороне базы данных.

Отложенное выполнение. LINQ-запросы в EF Core не отправляются в базу в момент составления — только когда запрашивается результат. Это позволяет наращивать запрос постепенно, и в базу уйдёт один итоговый SQL:

```
using var context = new AppDbContext();

// SQL ещё не выполнялся
IQueryable<Book> query = context.Books
    .Where(b => b.Price < 1000);
```

```
// Добавляем сортировку — SQL всё ещё не выполнялся
query = query.OrderBy(b => b.Title);

// Только здесь запрос уходит в базу
List<Book> books = await query.ToListAsync();
```

Запрос выполняется в момент вызова терминального метода: `ToListAsync()`, `ToArrayAsync()`, `FirstOrDefaultAsync()`, `SingleOrDefaultAsync()`, `CountAsync()`, `AnyAsync()`, `SumAsync()`, `MaxAsync()` и других.

Получение всех записей:

```
using var context = new AppDbContext();

var books = await context.Books.ToListAsync();
foreach (var b in books)
    Console.WriteLine($"[{b.Id}] {b.Title}");
```

`FindAsync` **— поиск по первичному ключу.** Сначала проверяет кэш контекста, и только если объект там не найден — идёт в базу:

```
using var context = new AppDbContext();

Book? book = await context.Books.FindAsync(1);
```

Фильтрация, сортировка и пагинация:

```
using var context = new AppDbContext();

int page     = 2;
int pageSize = 10;

var books = await context.Books
    .Where(b => b.Year >= 1900 && b.Price < 800) // WHERE
    .OrderBy(b => b.Title)                       // ORDER BY
    .ThenByDescending(b => b.Year)               // ORDER BY ... DESC
    .Skip((page - 1) * pageSize)                 // OFFSET
    .Take(pageSize)                              // LIMIT
    .ToListAsync();

// Общее количество для расчёта числа страниц
int total = await context.Books.CountAsync();
int pages = (int)Math.Ceiling((double)total / pageSize);
```

Проекция в DTO через `Select`:

DTO (Data Transfer Object) — это облегченная модель данных, предназначенная для передачи информации между слоями приложения или сервисами, которая содержит исключительно те поля, которые необходимы для выполнения конкретной задачи или запроса.

Вместо загрузки полного объекта можно выбрать только нужные поля. EF Core сгенерирует `SELECT` только для указанных столбцов:

```
using var context = new AppDbContext();

public record BookDto(int Id, string Title, string AuthorName, double
Price);

var dtos = await context.Books
    .Select(b => new BookDto(b.Id, b.Title, b.Author.Name, b.Price))
    .ToListAsync();
```

Загрузка связанных данных через `Include` и `ThenInclude`:

```
using var context = new AppDbContext();

// Один уровень — книги с авторами
var books = await context.Books
    .Include(b => b.Author)
    .ToListAsync();

// Несколько уровней — заказы с позициями и товарами
var orders = await context.Orders
    .Include(o => o.User)
    .Include(o => o.OrderItems)
        .ThenInclude(i => i.Product)
    .ToListAsync();

foreach (var order in orders)
{
    Console.WriteLine($"Заказ #{order.Id} — {order.User.Name}");
    foreach (var item in order.OrderItems)
        Console.WriteLine($"  {item.Product.Title} × {item.Qty}");
}
```

`Include` с фильтрацией — загружаем только часть связанных данных:

```
// Только активные книги каждого автора
var authors = await context.Authors
    .Include(a => a.Books.Where(b => b.Price < 500))
    .ToListAsync();
```

`AsNoTracking` — чтение без отслеживания:

Если данные нужны только для чтения и изменять их не планируется — отслеживание бесполезно и лишь тратит ресурсы. `AsNoTracking` его отключает:

```
using var context = new AppDbContext();

var books = await context.Books
    .AsNoTracking()
    .Include(b => b.Author)
    .ToListAsync();
```

Простое правило: методы, которые только читают и возвращают данные — добавляйте `AsNoTracking`. Методы, которые изменяют загруженные объекты — оставляйте отслеживание.

Агрегатные функции:

```
using var context = new AppDbContext();

int   total    = await context.Books.CountAsync();
int   cheap    = await context.Books.CountAsync(b => b.Price < 500);
bool  hasAny   = await context.Books.AnyAsync();
bool  exists   = await context.Books.AnyAsync(b => b.Title == "Идиот");

double maxPrice = await context.Books.MaxAsync(b => b.Price);
double minPrice = await context.Books.MinAsync(b => b.Price);
double avgPrice = await context.Books.AverageAsync(b => b.Price);
double sum      = await context.Books.SumAsync(b => b.Price);
```

Все вычисления выполняются на стороне базы данных — в память загружается только результат.

GroupBy — группировка:

```
using var context = new AppDbContext();

// Количество книг по годам
var byYear = await context.Books
    .GroupBy(b => b.Year)
    .Select(g => new { Year = g.Key, Count = g.Count() })
    .OrderBy(x => x.Year)
    .ToListAsync();

foreach (var item in byYear)
    Console.WriteLine($"{item.Year}: {item.Count} книг");
```

Ограничения транслятора LINQ.

Не все LINQ-выражения можно перевести в SQL. EF Core выбросит исключение `InvalidOperationException`, если встретит нетранслируемый вызов:

```
// Ошибка: произвольный метод C# нельзя транслировать в SQL
var books = await context.Books
    .Where(b => MyCustomFilter(b.Title))  // InvalidOperationException
    .ToListAsync();
```

Решение — разделить: сначала отфильтровать на стороне БД то, что можно, затем применить C#-логику в памяти:

```
// Сначала фильтруем в SQL то, что умеем
var books = await context.Books
    .Where(b => b.Price < 1000)
    .ToListAsync();

// Затем применяем произвольную C#-логику в памяти
var filtered = books
    .Where(b => MyCustomFilter(b.Title))
    .ToList();
```

Update

Обновление через отслеживание изменений — основной способ. Загружаем объект, меняем свойства, сохраняем. EF Core сам определяет, что именно изменилось, и обновляет только эти поля:

```
using var context = new AppDbContext();

var book = await context.Books.FindAsync(1);

if (book is not null)
{
    book.Price = 720;           // State → Modified
    book.Title = "Мастер и Маргарита (новое издание)";

    await context.SaveChangesAsync();
}
```

`ExecuteUpdateAsync` **— обновление без загрузки объектов.** Эффективен для массовых обновлений по условию:

```
using var context = new AppDbContext();

// Повысить цену всех книг дороже 2000 года на 10%
int updated = await context.Books
    .Where(b => b.Year < 2000)
    .ExecuteUpdateAsync(s => s
        .SetProperty(b => b.Price, b => b.Price * 1.1)
    );

Console.WriteLine($"Обновлено книг: {updated}");
```

`ExecuteUpdateAsync` не использует отслеживание изменений — он отправляет SQL напрямую. Это быстрее, но изменения не отражаются в уже загруженных объектах текущего контекста.

Delete

Удаление через `Remove`:

```
using var context = new AppDbContext();

var book = await context.Books.FindAsync(1);

if (book is not null)
{
    context.Books.Remove(book);      // State → Deleted
    await context.SaveChangesAsync(); // DELETE FROM Books WHERE Id = 1
}
```

`ExecuteDeleteAsync` **— удаление без загрузки объектов.** Значительно эффективнее для удаления по условию:

```
using var context = new AppDbContext();

// Удалить все книги дешевле 100 рублей
int deleted = await context.Books
    .Where(b => b.Price < 100)
    .ExecuteDeleteAsync();

Console.WriteLine($"Удалено книг: {deleted}");
```

6. Миграции

По мере развития приложения модели меняются: добавляются новые сущности, появляются новые поля, изменяются ограничения. Каждое такое изменение должно отразиться в структуре базы данных. Делать это вручную через SQL-скрипты неудобно и легко допустить рассинхронизацию между кодом и схемой. Для решения этой задачи в EF Core существуют **миграции**.

Миграция — это описание изменений структуры базы данных в виде C#-кода. Каждая миграция знает, как привести схему из предыдущего состояния в следующее (`Up`) и как откатить это изменение обратно (`Down`).

Создание первой миграции

Продолжаем с моделями `Author` и `Book` из предыдущих разделов. Контекст уже настроен, `EnsureCreated` из раздела 2 больше не подходит — переходим на миграции.

Создаём первую миграцию:

```
dotnet ef migrations add InitialCreate
```

EF Core сравнивает текущие модели с последним снимком схемы (пока его нет — сравнивает с пустой базой) и создаёт файлы миграции. В проекте появляется папка `Migrations`:

```
Migrations/
├── 20240115120000_InitialCreate.cs         — сама миграция
├── 20240115120000_InitialCreate.Designer.cs — служебные метаданные
└── AppDbContextModelSnapshot.cs            — снимок текущей модели
```

Применяем миграцию к базе данных:

```
dotnet ef database update
```

EF Core создаёт файл `library.db` и выполняет SQL из метода `Up`. Также создаётся служебная таблица `__EFMigrationsHistory`, в которую записывается имя каждой применённой миграции — при следующем `database update` EF Core проверяет её и применяет только новые миграции.

Изменение схемы

Добавим в `Book` новое поле — ISBN:

```
public class Book
{
    public int     Id       { get; set; }
    public int     AuthorId { get; set; }
    public string  Title    { get; set; } = "";
    public int     Year     { get; set; }
    public double  Price    { get; set; }
    public string? Isbn     { get; set; }  // новое поле — nullable
}
```

Создаём миграцию:

```
dotnet ef migrations add AddIsbnToBook
```

EF Core сравнивает модель со снимком и генерирует только изменения:

```
public partial class AddIsbnToBook : Migration
{
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name:     "Isbn",
            table:    "Books",
            type:     "TEXT",
            nullable: true);
    }

    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(name: "Isbn", table: "Books");
    }
}
```

Применяем:

```
dotnet ef database update
```

Добавление новой сущности работает так же — добавляем класс, регистрируем `DbSet` в контексте, создаём и применяем миграцию.

Ручная правка миграций. Иногда автоматически сгенерированной миграции недостаточно. При переименовании поля EF Core не умеет отличить переименование от удаления одного поля и добавления другого — он создаст `DropColumn` + `AddColumn`, что приведёт к потере данных. Такую миграцию нужно исправить вручную:

```
// EF Core сгенерирует это — данные потеряются:
migrationBuilder.DropColumn(name: "Title", table: "Books");
migrationBuilder.AddColumn<string>(name: "Name", table: "Books");

// Исправляем вручную — данные сохранятся:
migrationBuilder.RenameColumn(
    name:    "Title",
    table:   "Books",
    newName: "Name");
```

В миграцию также можно добавить произвольный SQL — например, заполнить новый столбец данными на основе существующих:

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.AddColumn<string>(
        name: "Slug", table: "Books", nullable: true);

    // Заполняем данными на основе существующего поля
    migrationBuilder.Sql("""
        UPDATE Books
        SET Slug = LOWER(REPLACE(Title, ' ', '-'))
        """);

    migrationBuilder.AlterColumn<string>(
        name: "Slug", table: "Books", nullable: false);
}
```

Откат миграций

Откатить базу до конкретной миграции — схема вернётся к состоянию после неё:

```
dotnet ef database update InitialCreate
```

Откатить все миграции — база вернётся к пустой схеме:

```
dotnet ef database update 0
```

При откате EF Core выполняет методы `Down` в обратном порядке — от последней миграции к указанной.

Удалить последнюю ещё не применённую миграцию (только файл, без изменения базы):

```
dotnet ef migrations remove
```

Применённые миграции не удаляются — только откатываются через `database update`. Удалять применённую миграцию вручную нельзя: это рассинхронизирует файлы и базу.

MigrateAsync vs EnsureCreated

В разделе 2 для создания базы использовался `EnsureCreatedAsync`. Разница между двумя подходами:

	`EnsureCreated`	`MigrateAsync`
Создаёт базу, если нет	✅	✅
Применяет миграции	❌	✅
Ведёт историю изменений	❌	✅
Поддерживает `database update`	❌	✅

`EnsureCreated` создаёт схему напрямую из моделей, минуя систему миграций. Если после этого создать миграцию и попробовать применить её — возникнет конфликт: таблицы уже существуют, но `__EFMigrationsHistory` пуста.

Для применения миграций из кода при старте приложения:

```
using var context = new AppDbContext();
await context.Database.MigrateAsync();
```

`MigrateAsync` применяет все ожидающие миграции и создаёт базу, если её нет.

7. Транзакции

В разделе 5 мы уже видели, что `SaveChangesAsync` выполняется атомарно — все накопленные изменения сохраняются либо целиком, либо откатываются при ошибке. Это называется **неявной транзакцией**. Но есть сценарии, где неявной транзакции недостаточно — для них существуют явные транзакции.

Неявные транзакции

Каждый вызов `SaveChangesAsync` автоматически оборачивается в транзакцию. Все изменения, накопленные в контексте к моменту вызова, применяются вместе или не применяются вовсе:

```
using var context = new AppDbContext();

// Три объекта добавлены в контекст
context.Authors.Add(new Author { Name = "Толстой",     Country = "Россия"
});
context.Authors.Add(new Author { Name = "Достоевский", Country = "Россия"
});
context.Authors.Add(new Author { Name = "Чехов",       Country = "Россия"
});

// Один SaveChangesAsync = одна транзакция
// Если любая вставка провалится — все три откатятся
await context.SaveChangesAsync();
```

Неявных транзакций достаточно для большинства простых операций. Они **не помогают** в двух случаях: когда нужно объединить несколько вызовов `SaveChangesAsync` в одну транзакцию, или когда между вызовами выполняются промежуточные проверки и при неудаче нужно откатить уже сделанные изменения:

```
using var context = new AppDbContext();

// Первый SaveChangesAsync — своя транзакция, данные уже в базе
var author = new Author { Name = "Булгаков", Country = "Россия" };
context.Authors.Add(author);
await context.SaveChangesAsync();

// Если здесь возникнет ошибка — author уже сохранён, откатить нельзя
var book = new Book { AuthorId = author.Id, Title = "Мастер и Маргарита",
Year = 1967, Price = 650 };
context.Books.Add(book);
await context.SaveChangesAsync(); // отдельная транзакция
```

Явные транзакции

Явная транзакция открывается через `BeginTransactionAsync` и охватывает все операции вплоть до `CommitAsync`. При ошибке вызывается `RollbackAsync`.

Общая структура:

```
using var context = new AppDbContext();
await using var transaction = await
context.Database.BeginTransactionAsync();

try
{
    // все операции внутри одной транзакции

    await context.SaveChangesAsync();
    await transaction.CommitAsync();
}
catch
{
    await transaction.RollbackAsync();
    throw;  // пробрасываем исключение дальше
}
```

`IDbContextTransaction` реализует `IAsyncDisposable`, поэтому используется `await using` — без него транзакция не освободит ресурсы корректно.

Пример: добавление автора с книгами

Добавим автора и его книги так, чтобы при любой ошибке в базе не осталось ни автора без книг, ни книг без автора:

```
public async Task<int> CreateAuthorWithBooksAsync(
    Author author,
    List<Book> books,
    CancellationToken token = default)
{
    using var context = new AppDbContext();
    await using var transaction = await
context.Database.BeginTransactionAsync(token);

    try
    {
        // Шаг 1: сохраняем автора — получаем author.Id
        context.Authors.Add(author);
        await context.SaveChangesAsync(token);
```

```
        // Шаг 2: проставляем AuthorId и сохраняем книги
        foreach (var book in books)
            book.AuthorId = author.Id;

        context.Books.AddRange(books);
        await context.SaveChangesAsync(token);

        // Оба SaveChangesAsync в одной транзакции — либо всё, либо ничего
        await transaction.CommitAsync(token);
        return author.Id;
    }
    catch
    {
        await transaction.RollbackAsync(token);
        throw;
    }
}
```

Использование:

```
try
{
    int authorId = await CreateAuthorWithBooksAsync(
        new Author { Name = "Булгаков", Country = "Россия" },
        new List<Book>
        {
            new Book { Title = "Мастер и Маргарита", Year = 1967, Price =
650 },
            new Book { Title = "Белая гвардия",      Year = 1925, Price =
490 }
        }
    );
    Console.WriteLine($"Автор Id={authorId} и его книги сохранены");
}
catch (Exception ex)
{
    Console.WriteLine($"Ошибка: {ex.Message}");
    // В базе ничего не осталось — ни автор, ни книги
}
```

Практическое задание

Что нужно сделать

Заменить хранение задач в файлах на реляционную базу данных SQLite через ORM Entity Framework Core. Весь остальной функционал консольного приложения должен остаться неизменным.

Структура проекта

Продолжайте работу в том же проекте. Внутри него создайте папку Data/ для контекста базы данных:

```
TodoList/
├── Commands/
├── Data/
│   └── AppDbContext.cs
├── Migrations/
├── Models/
│   ├── TodoItem.cs
│   └── Profile.cs
├── Services/
│   ├── TodoRepository.cs
│   └── ProfileRepository.cs
└── Program.cs
```

Что нужно сделать

1. Установить пакеты EF Core

```
dotnet add package Microsoft.EntityFrameworkCore
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package Microsoft.EntityFrameworkCore.Tools
dotnet tool install --global dotnet-ef
```

2. Настроить модель TodoItem и Profile для EF Core

Перенесите эти классы в папку Models.
Между Profile и TodoItem должна существовать связь один-ко-многим. Добавьте внешние ключи и навигационные поля в эти классы. С помощью атрибутов или Fluent API настройте:

- первичный и внешний ключ и поведение при удалении
- ограничения на поля (обязательное, максимальное и минимальное значение и т.п.)
- укажите поля и свойства, которые не должны сохраняться в БД

3. Создать DbContext

```
// Data/AppDbContext.cs
public class AppDbContext : DbContext
{
    public DbSet<TodoItem> Todos => Set<TodoItem>();
    public DbSet<Profile> Profiles => Set<Profile>();

    protected override void OnConfiguring(DbContextOptionsBuilder o)
        => o.UseSqlite("Data Source=todos.db");
}
```

4. Создать репозиторий TodoRepository и ProfileRepository

Вынесите всю логику работы с базой данных для задач в отдельный класс `TodoRepository`. Реализуйте методы:

- `GetAll()` — вернуть все задачи
- `Add(TodoItem item)`
- `Update(TodoItem item)`
- `Delete(int id)`
- `SetStatus(int id, TodoStatus status)`

Создайте аналогичный класс для профилей пользователей `ProfileRepository`.

Репозитории должны находиться в папке Services.

Используйте `using var context = new AppDbContext()` в каждом методе репозитория. Контекст должен жить коротко — создаваться и освобождаться в рамках одной операции.

5. Создать и применить миграцию

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

6. Обновить команды

Замените прямые обращения к `FileManager` на вызовы методов `TodoRepository` и `ProfileRepository`. Класс `FileManager` удалите или пометьте как устаревший.

Требования к сдаче

- Задачи сохраняются в файл `todos.db` (SQLite)
- После перезапуска программы задачи загружаются из базы
- Файл `FileManager.cs` удалён или не используется для Todo

- Создана хотя бы одна миграция через `dotnet ef`
- Коммиты после каждого логического изменения
- Обновлено `README.md`, в котором упомянуто переключение на БД